

Lab 2: Writing Your First *Controller*

In Lab 1, you generated a Spring Boot project and saw the Whitelabel Error Page. Now it's time to fix that — you'll write a controller that handles HTTP requests and returns real responses in your browser.

- ▶ Fresh project from `start.spring.io`
- ▶ You will write code in this lab
- ▶ Type it manually – don't copy-paste

01 What We're Building

By the end of this lab, your `hello-controller` project will have:

- A REST controller that responds to HTTP GET requests
- An endpoint that returns plain text
- An endpoint that uses `@PathVariable` (value from the URL path)
- An endpoint that uses `@RequestParam` (value from the query string)
- An endpoint that returns **JSON** by returning a Java object

No more Whitelabel Error Page — your app will actually serve content.

Remember from the Spring Boot lesson: A controller is the **first layer** of your application — it receives HTTP requests and returns responses. In Spring Boot, we use `@RestController` to mark a class as a REST controller and `@GetMapping` to map a URL to a method.

02 Generate a New Project from Spring Initializr

Just like Lab 1, we start at <https://start.spring.io> . Practice makes perfect — the more you use Spring Initializr, the faster you'll be at setting up new microservices.

✓ DO THIS NOW

Open your browser and go to <https://start.spring.io>

Fill In the Settings

FIELD	VALUE
Project	Maven
Language	Java
Spring Boot	Latest stable (no SNAPSHOT/M/RC)
Group	<code>com.example</code>
Artifact	<code>hello-controller</code>
Package name	<code>com.example.hellocontroller</code> (verify — no hyphens!)
Packaging	Jar
Java	17 or 21 (match your installed JDK)
Configuration	Properties

Add ONE Dependency

✓ DO THIS NOW

Click **"ADD DEPENDENCIES"** (or Ctrl+B) → search **"Spring Web"** → click to add.

You should see **only 1 dependency: Spring Web**.

⚠️ **Only Spring Web!** Do NOT add JPA, MySQL, or anything else. Same as Lab 1 — keep it simple.

Generate, Download, and Open

✓ DO THIS NOW

- 1 **Click GENERATE** (or Ctrl+Enter) — `hello-controller.zip` will download
- 2 **Extract the ZIP file**
- 3 **Open in IntelliJ** — File → Open → select the `hello-controller` folder (make sure you see `pom.xml` inside!). Or right-click `pom.xml` → Open with → IntelliJ IDEA.
- 4 **Wait for Maven to finish** — watch the progress bar at the bottom

Verify It Runs

✓ DO THIS NOW

Run `HelloControllerApplication.java` (the generated main class) — right-click → Run.

You should see the Spring Boot banner and `Started HelloControllerApplication` in the console.

Visit <http://localhost:8080> in your browser — you should see the **Whitelabel Error Page** (no controllers yet — that's expected!).

Stop the app (■ button) before continuing.

Good — you've done this before! The process is the same as Lab 1: generate → download → extract → open → run → see Whitelabel. You're already getting faster at it. Now let's add a controller to fix that 404.

03 Create Your First Controller

Step 1: Create the Controller Package

✓ DO THIS NOW

In IntelliJ, find `com.example.hellocontroller` under `src/main/java` .
Right-click on it → **New** → **Package** → type `controller` → press Enter.

You should now have: `com.example.hellocontroller.controller`

Why a separate package? We organize code by layer: controllers in `controller/`, services in `service/`, models in `model/`. This keeps things clean as the project grows. Spring's `@ComponentScan` (part of `@SpringBootApplication`) automatically finds classes in sub-packages — so the controller will be detected without any extra configuration.

Step 2: Create the HelloController Class

✓ DO THIS NOW

Right-click on the `controller` package → **New** → **Java Class** → type `HelloController` → press Enter.

Step 3: Write Your First Endpoint

✓ TYPE THIS CODE (DON'T COPY-PASTE)

Replace everything in `HelloController.java` with this code. **Type it manually** — this helps you remember the annotations and structure.

JAVA — HELLOCONTROLLER.JAVA

```
package com.example.hellocontroller.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Spring Boot!";
    }
}
```

⚠ **Seeing red errors on `@RestController` or `@GetMapping`?** This means the import is missing or wrong. Make sure these two import lines are at the top of your file:

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

Quick fix in IntelliJ: Click on the red underlined annotation → press **Alt+Enter** (Windows) or **Option+Enter** (Mac) → select **"Import class"**. Make sure you pick the one from

`org.springframework.web.bind.annotation` — not from a different package. If you see multiple options, always choose the one that starts with `org.springframework`.

⚠️ **Import each class specifically — do NOT use asterisk (*) imports!**

```
// ✅ CORRECT — import each class by name
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

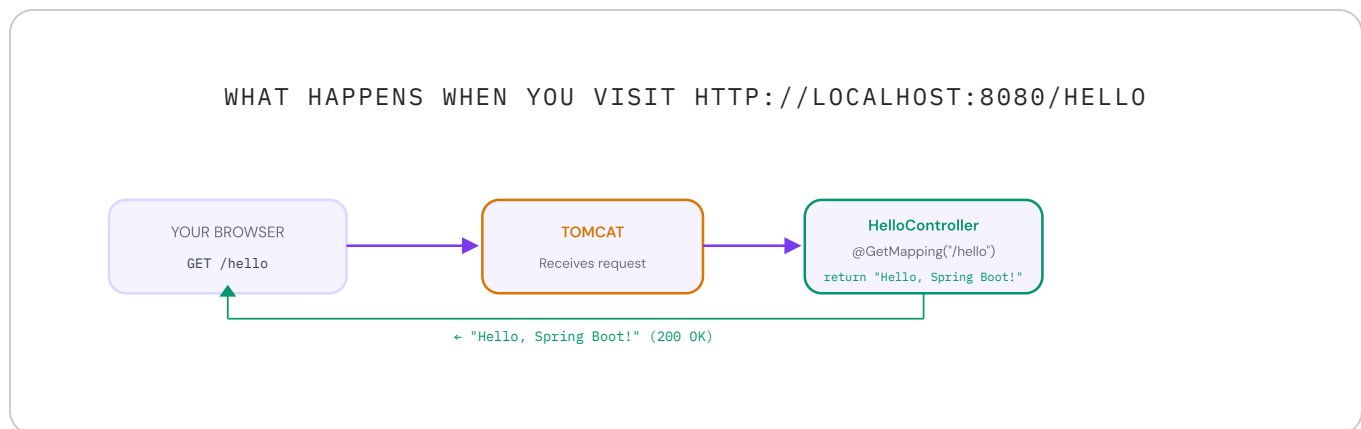
// ❌ WRONG — do NOT use wildcard/asterisk imports
import org.springframework.web.bind.annotation.*;
```

Why? Wildcard imports (`.*`) import *every* class in the package — even ones you don't use. This causes problems: (1) you can't tell which classes your code actually uses, (2) it can lead to name conflicts if two packages have classes with the same name, (3) it makes the code harder to read and maintain. Always import the **specific class** you need.

That's it — 12 lines of code. Let's understand **every single line**:

LINE	WHAT IT DOES
<code>package</code> <code>com.example.hellocontroller.controller;</code>	This class belongs to the <code>controller</code> sub-package. Verify this matches your folder structure.
<code>import ...GetMapping;</code>	Imports the annotation for handling GET requests. Your IDE may add this automatically as you type.
<code>import ...RestController;</code>	Imports the annotation that marks this as a REST controller.
<code>@RestController</code>	Tells Spring three things: (1) this class is a Bean — create and manage it, (2) it handles HTTP requests , (3) every method returns data directly (not an HTML view).

LINE	WHAT IT DOES
	Remember: <code>@RestController</code> = <code>@Controller</code> + <code>@ResponseBody</code> .
<code>public class HelloController</code>	A regular Java class. Name it descriptively — always end controller class names with "Controller".
<code>@GetMapping("/hello")</code>	When someone sends a GET request to the URL <code>/hello</code> , run this method. This maps a URL path to a Java method.
<code>public String sayHello()</code>	A regular Java method that returns a String. The method name is your choice — make it descriptive.
<code>return "Hello, Spring Boot!";</code>	This String becomes the HTTP response body . The browser will display this text.



 **REST API Connection:** `@GetMapping("/hello")` = HTTP **GET** method + `/hello` endpoint. This is exactly the request-response cycle from the REST lesson. The browser sends a GET request, the controller handles it, and returns a response with status 200 OK.

04 Run and Test Your Controller

✓ DO THIS NOW

Run your application: open `HelloControllerApplication.java` → right-click → **Run**

Wait for `Started HelloControllerApplication` in the console.

If You See: APPLICATION FAILED TO START

Some of you might see this error instead of the Spring Boot banner:

CONSOLE — ERROR

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

Web server failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080
or configure this application to listen on another port.

This means **another application is already using port 8080** on your computer. This often happens when:

- Your **Lab 1 project** is still running in the background — you forgot to stop it
- Another Spring Boot app or web server is using port 8080
- A program like Skype, XAMPP, or another tool is occupying that port

How to Fix It

OPTION	HOW
Option 1: Stop the other app	Check your IntelliJ tabs — if Lab 1's app is still running, click the red ■ stop button on it. Then try running Lab 2 again.
Option 2: Change your port	Open <code>application.properties</code> and add: <code>server.port=8081</code> Your app will now run on <code>http://localhost:8081</code> instead. Use this URL for all tests in this lab.
Option 3: Kill the process (Windows)	Open Command Prompt and run: <code>netstat -ano findstr :8080</code> Find the PID number, then run: <code>taskkill /PID <number> /F</code>
Option 3: Kill the process (Mac/Linux)	Open Terminal and run: <code>lsof -i :8080</code> Find the PID, then run: <code>kill -9 <PID></code>

Easiest fix: Option 1 — just stop your Lab 1 app. If that doesn't work, Option 2 — add `server.port=8081` to your `application.properties`. You'll be up and running in seconds.

✓ NOW OPEN YOUR BROWSER

Go to:

`http://localhost:8080/hello`

What You Should See

BROWSER OUTPUT

Hello, Spring Boot!

Congratulations! No more Whitelabel Error Page. Your controller is working. That text traveled through: browser → HTTP GET request → Tomcat → Spring MVC → `HelloController.sayHello()` → HTTP response → back to browser.

Try the Root URL Too

✓ TRY THIS

Go to `http://localhost:8080/` (just the root, no `/hello`)

You'll still see the **Whitelabel Error Page** on `/` because we only mapped `/hello` . That's expected — the 404 means "no controller is mapped to this URL." Your `/hello` endpoint works, but `/` does not (yet).

Stop the app before continuing. Press the red ■ stop button in IntelliJ. We'll make changes and restart.

05 Dynamic URLs with `@PathVariable`

Right now, `/hello` always returns the same text. What if we want to say hello to a **specific person**? We use `@PathVariable` — it extracts a value from the URL path itself.

✓ ADD THIS METHOD TO HELLOCONTROLLER.JAVA

Open `HelloController.java` and add this method **inside the class**, after the `sayHello()` method. Don't forget to add the import at the top.

JAVA – ADD THIS METHOD INSIDE HELLOCONTROLLER CLASS

```
@GetMapping("/hello/{name}")
public String sayHelloTo(@PathVariable String name) {
```

```
    return "Hello, " + name + "!";  
}
```

Red underline on @PathVariable? Your IDE doesn't know what it is yet — it needs to import the class. Click on the red text → press **Alt+Enter** (Windows) or **Option+Enter** (Mac) → select **"Import class"**. Choose `org.springframework.web.bind.annotation.PathVariable`. Your IDE will add the import line at the top automatically. Do this every time you see a red underline on an annotation.

Let's understand what's new:

PART	WHAT IT DOES
<code>"/hello/{name}"</code>	The <code>{name}</code> part is a placeholder . Whatever value appears in that position of the URL gets captured. <code>/hello/Juan</code> → <code>name = "Juan"</code> .
<code>@PathVariable</code> <code>String name</code>	Tells Spring to take the value from <code>{name}</code> in the URL and put it into this <code>name</code> parameter. The variable name must match the placeholder name.

✅ **RUN AND TEST**

Restart the app and try these URLs in your browser:

URL	EXPECTED RESPONSE
<code>http://localhost:8080/hello</code>	<code>Hello, Spring Boot!</code> (original endpoint still works)
<code>http://localhost:8080/hello/Juan</code>	<code>Hello, Juan!</code>
<code>http://localhost:8080/hello/Maria</code>	<code>Hello, Maria!</code>

URL	EXPECTED RESPONSE
<code>http://localhost:8080/hello/SpringBoot</code>	Hello, SpringBoot!

Notice: Both `/hello` and `/hello/{name}` work. Spring is smart enough to route `/hello` to `sayHello()` and `/hello/Juan` to `sayHelloTo()` based on the URL pattern.

06 Query Parameters with @RequestParam

Another way to pass data is through **query parameters** — the part after the `?` in a URL. This is common for filtering, searching, and optional values.

✅ ADD THIS METHOD TO HELLOCONTROLLER.JAVA

Add the import and method:

JAVA – ADD THIS METHOD INSIDE HELLOCONTROLLER CLASS

```
@GetMapping("/greet")
public String greet(@RequestParam(defaultValue = "World") String name) {
    return "Greetings, " + name + "!";
}
```

Red underline on @RequestParam? Same fix — click on it → **Alt+Enter** → **Import class** → choose `org.springframework.web.bind.annotation.RequestParam`.

PART	WHAT IT DOES
<code>@RequestParam</code> <code>String name</code>	Reads the value of <code>name</code> from the query string. For URL <code>/greet?name=Juan</code> , <code>name = "Juan"</code> .

PART	WHAT IT DOES
<pre>defaultValue = "World"</pre>	If no <code>name</code> parameter is provided (<code>/greet</code> without <code>?name=...</code>), use <code>"World"</code> as the default. Without this, calling <code>/greet</code> without a parameter would cause a <code>400 Bad Request</code> error.

✓ RUN AND TEST

Restart the app and try these URLs:

URL	EXPECTED RESPONSE	WHY
<code>http://localhost:8080/greet</code>	Greetings, World!	No <code>?name=</code> → uses <code>defaultValue</code>
<code>http://localhost:8080/greet?name=Juan</code>	Greetings, Juan!	<code>name</code> parameter provided
<code>http://localhost:8080/greet?name=Spring Boot</code>	Greetings, Spring Boot!	Spaces are OK in query params

@PathVariable vs @RequestParam — Quick Reminder

ASPECT	@PATHVARIABLE	@REQUESTPARAM
Value comes from	The URL path : <code>/hello/Juan</code>	The query string : <code>/greet?name=Juan</code>
Use for	Identifying a specific resource (by ID or name)	Filtering, sorting, optional values
Required by default?	Yes — it's part of the URL	Yes, unless you set <code>defaultValue</code>

07 Return JSON — Like a Real API

So far we've returned plain text. But real REST APIs return **JSON**. The amazing thing about Spring Boot: you just return a Java object, and it **automatically converts it to JSON**. You don't write any JSON yourself.

Step 1: Create a Model Class

✓ DO THIS NOW

Right-click on `com.example.hellocontroller` → **New** → **Package** → name it `model`

Right-click on `model` → **New** → **Java Class** → name it `Greeting`

✓ TYPE THIS CODE

JAVA — GREETING.JAVA

```
package com.example.hellocontroller.model;

public class Greeting {

    private String message;
    private String from;
    private int year;

    // Constructor
    public Greeting(String message, String from, int year) {
        this.message = message;
        this.from = from;
        this.year = year;
    }

    // Getters — Jackson needs these to build the JSON!
    public String getMessage() {
        return message;
    }
}
```

```

    }

    public String getFrom() {
        return from;
    }

    public int getYear() {
        return year;
    }
}

```

Let's understand this class:

PART	WHAT IT DOES
Fields (<code>message</code> , <code>from</code> , <code>year</code>)	The data this object holds. These become the JSON keys.
Constructor	How we create a new Greeting object with values.
Getters (<code>getMessage()</code> , etc.)	Methods that return each field's value. Jackson reads these to build the JSON output.

⚠️ **Getters are required!** Jackson (the JSON library included in Spring Web) uses getter methods to know what to include in the JSON. The naming convention matters: for a field called `message`, the getter must be `getMessage()`. If you forget a getter, that field will **not appear** in the JSON response. No getter = invisible field.

Why are we writing getters manually? In real projects, many developers use Lombok (`@Data`) to auto-generate getters. But we're skipping Lombok for now so you understand exactly what's happening. Once you're comfortable writing getters yourself, you'll appreciate what Lombok does for you later.

Step 2: Add a JSON Endpoint to the Controller

✓ ADD THIS METHOD TO HELLOCONTROLLER.JAVA

JAVA – ADD THIS METHOD INSIDE HELLOCONTROLLER CLASS

```
@GetMapping("/api/greeting")
public Greeting getGreeting() {
    return new Greeting("Hello from the API!", "Spring Boot", 2026);
}
```

Red underline on Greeting? Same fix — **Alt+Enter** → **Import class** → choose `com.example.hellocontroller.model.Greeting`. This is the class you just created in the `model` package.

Notice the difference: this method returns a `Greeting` **object**, not a `String`. Spring Boot sees you're returning an object from a `@RestController` and automatically calls Jackson to convert it to JSON.

✓ RUN AND TEST

Restart the app and go to:

`http://localhost:8080/api/greeting`

What You Should See

BROWSER OUTPUT (JSON)

```
{
  "message": "Hello from the API!",
  "from": "Spring Boot",
  "year": 2026
}
```

This is JSON! You returned a Java object, and Spring Boot automatically converted it. This is exactly how real REST APIs work — every microservice you build will return JSON like this.

What just happened behind the scenes:

1. Your `getGreeting()` method created a `Greeting` Java object
2. Because the class is marked with `@RestController`, Spring knows to convert the return value to JSON
3. Jackson reads the getter methods: `getMessage()` → "message" key, `getFrom()` → "from" key, `getYear()` → "year" key
4. Jackson builds the JSON string and Spring sends it as the HTTP response with `Content-Type: application/json`

 **REST API Connection:** Remember from the REST lesson — responses include a **status code** (200 OK), **headers** (`Content-Type: application/json`), and a **body** (the JSON). Spring Boot handled all three for you automatically.

08 Lab 2 Recap

Your Final Project Structure

WHAT YOU BUILT

```
▼ hello-controller
  ▼ src/main/java/com.example.hellocontroller
    HelloControllerApplication.java      ← entry point (generated)
    ▼ controller
      HelloController.java              ← you wrote this!
    ▼ model
      Greeting.java                    ← you wrote this!
  ▼ src/main/resources
    application.properties
    pom.xml
```

Your Endpoints

URL	RETURNS	ANNOTATION USED
<code>/hello</code>	Plain text	<code>@GetMapping</code>
<code>/hello/{name}</code>	Dynamic text	<code>@GetMapping</code> + <code>@PathVariable</code>
<code>/greet?name=X</code>	Text with query param	<code>@GetMapping</code> + <code>@RequestParam</code>
<code>/api/greeting</code>	JSON object	<code>@GetMapping</code> + Java object → Jackson → JSON



What you learned in this lab

1. How to create a package and a class inside your Spring Boot project
2. How `@RestController` marks a class as a REST controller (Bean + HTTP handler + returns data)
3. How `@GetMapping("/path")` maps a URL to a Java method
4. How `@PathVariable` extracts values from the URL path (`/hello/Juan`)
5. How `@RequestParam` extracts values from query strings (`/greet?name=Juan`)
6. How to return JSON by returning a Java object — Jackson converts it automatically
7. Why getters are required — Jackson reads them to build the JSON keys and values
8. The complete request flow: browser → Tomcat → Spring MVC → your controller → response

Submission



SUBMIT YOUR LAB

Compress your project folder into a ZIP file and submit it using the following naming format:

Lab2_EID_School.zip

Replace **EID** with your Accenture email (without @accenture.com) and **School** with your school name.

Example: `Lab2_juan.dela.cruz_TechUniv.zip`

⚠ **Before zipping:** Make sure you zip the `hello-controller` folder that contains `pom.xml` at the root. When someone unzips your file, they should be able to open it in IntelliJ and run it immediately.



Next — Lab 3: Adding a Database

In the next lab, we'll add **Spring Data JPA** and a **database** so our API can store and retrieve real data. We'll build a complete Product Service with CRUD operations (Create, Read, Update, Delete).

The learning path: Microservices → REST API → Spring Framework → Spring Ecosystem → Spring Boot → Spring Initializr → Lab 1 → **Lab 2: First Controller (today)** → Lab 3: Adding a Database (next)